

## Resumen Conciso — Entrevista Técnica Java/Spring/Microservicios

### Java Core

- **HashMap vs ConcurrentHashMap:** HashMap no es thread-safe; ConcurrentHashMap sí, usando bloqueos finos por bucket (sin bloquear todo el mapa).
- **Interfaz vs clase abstracta:** interfaz = contrato (herencia múltiple, sin estado); clase abstracta = jerarquía compartida (una sola herencia, puede tener estado y constructores).
- **Garbage Collector:** libera memoria de objetos no alcanzables automáticamente. Ciclo generacional: Young Gen (Minor GC frecuente) → Old Gen (Major GC). Mark → Sweep → Compact.
- **== vs .equals():** == compara referencias (o valores primitivos); .equals() compara contenido lógico (sobrescribible).
- **Colecciones:** List = ordenada con duplicados; Set = únicos; Map = clave-valor. Elige según necesites orden/índice, unicidad, o búsqueda por clave.
- **Programación funcional:** tratar funciones como valores (lambdas, interfaces funcionales, streams), favoreciendo inmutabilidad y composición.
- **Lambdas:** código conciso, expresan intención, habilitan Streams, pasan comportamiento como parámetro.
- **Streams:** procesamiento declarativo de colecciones (filter/map/collect), operaciones lazy, paralelizables.
- **Inmutabilidad:** objeto cuyo estado no cambia tras crearse (final, sin setters). Da thread-safety y previsibilidad.
- **Java 8/17/21:** 8 = lambdas, streams, Optional, java.time. 17 = records, sealed classes, pattern matching, text blocks. 21 = Virtual Threads, pattern matching en switch, sequenced collections.

### Spring Boot

- **IoC:** el framework crea y gestiona las dependencias, no el objeto mismo (Dependency Injection).
- **@Component/@Service/@Repository:** todas son beans; @Service = lógica de negocio, @Repository = acceso a datos (+ traducción de excepciones).
- **Spring Boot:** simplifica Spring con auto-configuración, starters, servidor embebido y Actuator.
- **Config por ambiente:** perfiles (application-{profile}.yaml) + variables de entorno para secretos.
- **Spring Security:** framework de autenticación/autorización con filtros de seguridad.
- **JWT:** login → generar token firmado con claims → cliente lo envía en header → filtro lo valida en cada request.

- **Spring Data JPA:** repositorios como interfaces, sin implementar CRUD manualmente.
- **CrudRepository vs JpaRepository:** JpaRepository extiende CrudRepository, agrega paginación, batch, y ordenamiento.
- **@Profile:** activa beans/configuración según el ambiente activo.
- **Manejo global de excepciones:** @RestControllerAdvice + @ExceptionHandler centralizado.

## Microservicios

- **Ventajas vs monolito:** despliegue independiente, escalado selectivo, aislamiento de fallos, libertad tecnológica.
- **Patrones de comunicación:** REST/gRPC síncrono, eventos (Kafka/RabbitMQ), Saga, Circuit Breaker, API Gateway, Service Mesh.
- **Síncrono vs asíncrono:** síncrono espera respuesta (acopla disponibilidad); asíncrono desacopla vía mensajería (consistencia eventual).
- **API Gateway:** punto de entrada único (ruteo, auth, rate limiting).
- **Service Discovery:** los servicios se encuentran dinámicamente sin IPs hardcodeadas (Eureka, Consul, DNS de K8s).
- **Transacciones distribuidas:** patrón Saga (coreografía u orquestación) en lugar de 2PC.
- **Circuit Breaker:** corta llamadas a un servicio fallando (closed → open → half-open) para evitar efecto cascada.
- **Resilience4j:** librería de resiliencia (circuit breaker, retry, rate limiter, bulkhead) para Spring Boot.
- **Observabilidad:** logs estructurados con trace ID + métricas (Actuator/Micrometer + Prometheus) + tracing distribuido (Zipkin/Jaeger).
- **Métricas en producción:** latencia (p95/p99), tasa de errores, throughput, uso de CPU/memoria/GC, saturación de pools.

## APIs REST

- **PUT vs PATCH:** PUT reemplaza el recurso completo; PATCH aplica cambios parciales. Ambos idempotentes.
- **Códigos HTTP frecuentes:** 200, 201, 204, 400, 401, 403, 404, 409, 500.
- **Versionado de API:** por URL (/v1/), por header, o por query param — URL es lo más común.
- **Idempotencia:** repetir la misma operación da el mismo resultado final (GET/PUT/DELETE lo son por naturaleza; POST se hace idempotente con una key).
- **Seguridad de API:** HTTPS, OAuth2/JWT, validación de entrada, rate limiting, CORS correcto, sin exponer stack traces.
- **OpenAPI/Swagger:** especificación estándar para describir APIs + herramientas para documentación interactiva.
- **Documentación:** OpenAPI auto-generado desde código + README + ejemplos reales de request/response.

## Bases de Datos

- **SQL vs NoSQL:** SQL = esquema fijo, relaciones, ACID; NoSQL = esquema flexible, escalabilidad horizontal, consistencia eventual.
- **Índice:** estructura (B-Tree) que acelera búsquedas; úsalo en columnas de WHERE/JOIN/ORDER BY con alta cardinalidad.
- **Niveles de aislamiento:** Read Uncommitted → Read Committed → Repeatable Read → Serializable (de menos a más estricto).
- **Consulta N+1:** 1 query para la lista + N queries adicionales por cada relación (se arregla con JOIN FETCH/@EntityGraph).
- **Optimizar consulta lenta:** revisar plan de ejecución, agregar índices, evitar SELECT \*, paginar, evitar N+1.
- **JOIN/LEFT JOIN/INNER JOIN:** INNER = solo coincidencias; LEFT = todas las filas de la izquierda aunque no haya match en la derecha.
- **PostgreSQL/SQL Server/MongoDB:** (personal) tu experiencia fuerte es en SQL Server (T-SQL, stored procedures) y MariaDB/MySQL reciente.

## Kafka / Mensajería

- **Apache Kafka:** plataforma de streaming de eventos distribuida, log particionado y replicado de alto throughput.
- **Queue vs Topic:** Queue = un mensaje, un consumidor (se elimina al procesarse); Topic = múltiples consumidores/grupos independientes leen el mismo log.
- **Consumer Group:** grupo de consumidores que se reparten las particiones de un topic para paralelizar el consumo.
- **Garantizar entrega:** acks=all, réplicas, commit de offset después de procesar, o exactly-once con transacciones.
- **Evitar duplicados:** idempotencia en productor (enable.idempotence) y en consumidor (deduplicación por messageId).
- **RabbitMQ/IBM MQ:** (personal) completar según tu experiencia real.

## Docker y Kubernetes

- **Docker:** empaqueta app + dependencias en contenedores portables y consistentes entre entornos.
- **Imagen vs contenedor:** imagen = plantilla inmutable; contenedor = instancia en ejecución de una imagen.
- **Optimizar imagen:** base ligera (alpine), multi-stage build, minimizar capas, .dockerignore, cachear dependencias.
- **Kubernetes:** orquestador de contenedores que automatiza despliegue, escalado y auto-recuperación.
- **Pod:** unidad mínima de despliegue; uno o más contenedores que comparten red/almacenamiento.
- **Deployment:** gestiona réplicas de Pods declarando el estado deseado (imagen, cantidad, estrategia de update).

- **ConfigMap:** configuración externa desacoplada de la imagen (variables/archivos); datos sensibles van en Secret.
- **Rollout/Rollback:** `kubectl set image + RollingUpdate` sin downtime; `kubectl rollout undo` para revertir a la revisión anterior.